Reinforcing learning: DQN

YX Zhao
BJTU

October 30, 2025

Contents

1. Concept

2. RL theory

3. DQN

Concept

Concept

This section provides a clean, professional understanding of concept in Reinforceing learning.

Key concept:

- RL model
- components of RL, such as agent, env, state, action, reward, return, MDP

RL model

Two components

agent envoroment

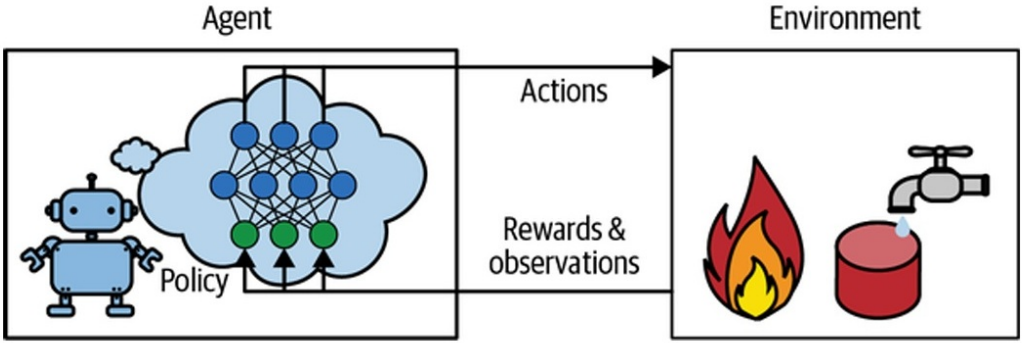


Figure 18-2. Reinforcement learning using a neural network policy

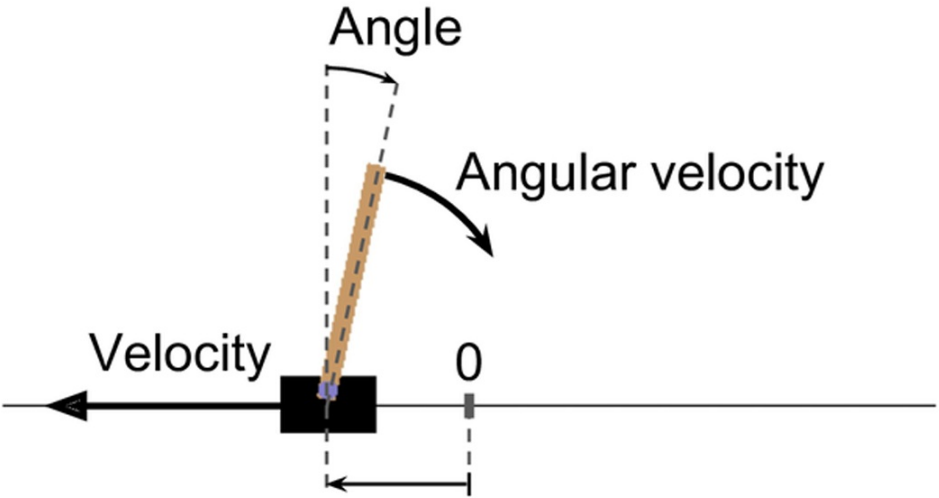
Figure: RL model

Example

- Cartpole
- programe play video game
- programe play chess
- autodrive
- rebot in maze

Cart pole

cartpole is minist of RL



episode

episode is a round of agent and env interactions

- play chess untill win or lose
- play video game untill win or lose
- rebot in maze, robot arrive at destination

key 1

we'll probably collect many episodes of data in order to train an oral agen.

key 2

Some env go on forever. such as stock market. We make it episodic by define an episode as a fixed number of interaction.

state

action is what the agent does as in its environment. action space is the set of all possible actions in that environment. .

- In a maze game, the action could be up and down.
- In a chess game, the action space is all possible in a chess move.
- In auto drive, action is [The steering wheel angle,Throttle pedal pressure/position]

Action

key 2

state can be combination of things observed in in current step and last few steps..

state space

State space is set of all possible state.
what it is if state is continuous variable?

Reward

Reward

Reward is a scaler that env gives to the agent after each step. By this value, the agent knows how well it is doing. You can think of it like training an animal to reinforce its behavior. When you train a dog to sit, if it sits successfully and you give it a treat, if it does not sit successfully, then you do not give it a treat.

Key1

reinforcement learning rewards are not necessarily a positive thing. So if you get a negative reward that's generally not a good thing. The key is that the agent can still use the rewards whether they are positive or negative to modify its actions.

Key2

the goal of an agent is to maximize or to achieve maximum reward over each episode.

Policy

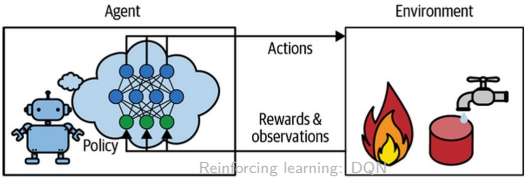
Policy

Policy is a function map state to action, and it describe the behaviors of agent. According to Policy, action is dependent only on current state and is independent on provevious state.

Two kind of policy

.

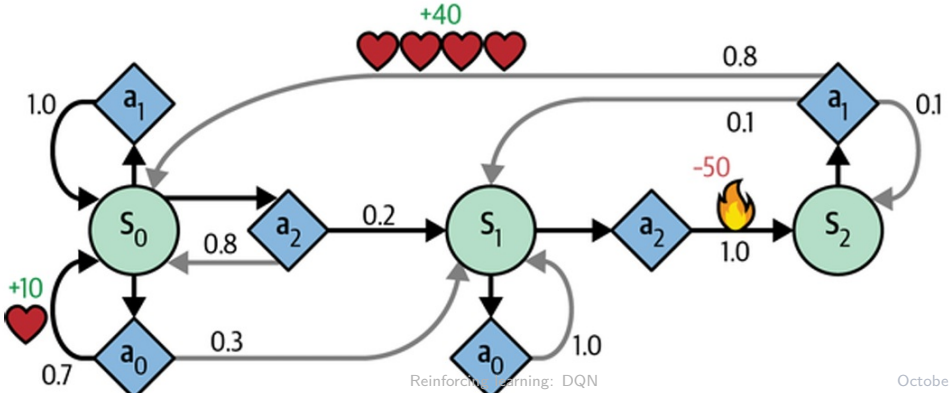
- determinic policy. $a = \pi(s)$.
- probalistic policy.$\pi(a|s)$ is a probabilitic distribution. i.e, in card game, choose king and queen with 0.9 and 0.1 respectively. prabability



State dynamic

MDP

State dynamic can be described as markov decision process(MDP).
state transition probability is describe is $p(s', r|s, a)$



MDP model

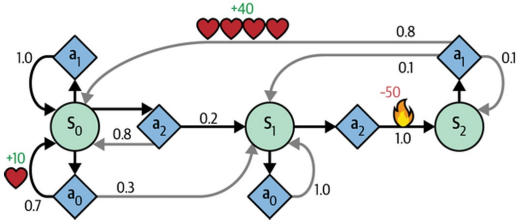


Figure 18-8. Example of a Markov decision process

Figure: MDP

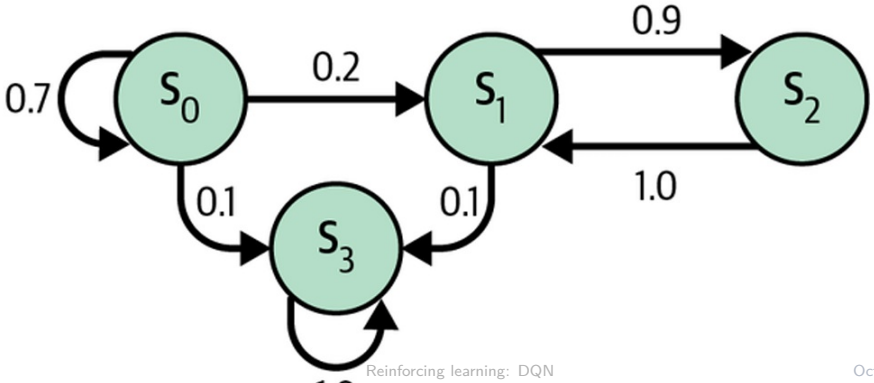
```
transition_probabilities = [  # shape=[s, a, s']
    [[0.7, 0.3, 0.0], [1.0, 0.0, 0.0], [0.8, 0.2, 0.0]],
    [[0.0, 1.0, 0.0], None, [0.0, 0.0, 1.0]],
    [None, [0.8, 0.1, 0.1], None]
]
rewards = [  # shape=[s, a, s']
    [[+10, 0, 0], [0, 0, 0], [0, 0, 0]],
    [[0, 0, 0], [0, 0, 0], [0, 0, -50]],
    [[0, 0, 0], [+40, 0, 0], [0, 0, 0]]
]
possible_actions = [[0, 1, 2], [0, 2], [1]]
```

Figure: MDP

Markov process vs MDP

Markov process

There is no action and reward in markov process. markov process. It merely described the evolution of the state without exercising control.
State transition probability is described is $p(s'|s)$



key of MDP

It is a model

MDP is a model to describe env and it assume the next state is only dependent on current state and the action you chose.
This assumption is not necessary realstic, but it works.

We don't konw transition probability

In actually work, we usually don't know $p(s', r|s, a)$.

Return

definition

agent try to maximize future rewards. The past rewards have been given and we have nothing to do with them. But we can chose best action in the future to maximize return which is sum of future rewards.

return at t step

The return at t step $G(t)$: $G(t) = r(t) + r(t+1) + \cdots + r(T)$
Discount return: $G(t) = r(t+1) + \gamma r(t+2) + \gamma^2 r(t+3) + \cdots + \gamma^{T-t-1} r(T)$. money is less valuable in future because inflate rate.

RL theory

Concept

This section provides a quantitative analysis of MDP.

Content:

- value functions
- bellman equation
- optimal policy

$V_\pi(s)$ and bellman equation

basic idea

Both The action sequence is undetermic due to probabity policy and state transition is probabilutic, The trajectory of rewards is stochastic, hence the return is also random. so we cannot maximize value of returen directly. Instead, we try to maximize expectation of return.

state value function $V_\pi(s)$ given a policy π

$V_\pi(s) = E_\pi[G(t)|s(t) = s]$.

bellman equation

expand above $V_\pi(s)$, we have $V_\pi(s) = E_\pi[r(t+1) + \gamma V_\pi(s(t+1))|s(t) = s]$.
$V(s)$ is recursive which mean it dependent on itself. and we can solve this equation recursively.

$V_\pi(s)$ and bellman equation

$G(t)$ is also resursive

$G(t) = r(t+1) + \gamma r(t+2) + \gamma^2 r(t+3) + \cdots + \gamma^{T-t-1} r(T)$
$G(t) = r(t+1) + \gamma G(t+1)$, this make bellman equation sense

expand expectation of $V_\pi(s)$

$$V_\pi(s) = \sum_a \sum_{s'} \sum_r \pi(a|s) p(s', r|s, a)(r + \gamma V_\pi(s'))$$

Q value

$$Q_\pi(s,a) = E_\pi[G(t)|s(t)=s, A(t)=a] \tag{1}$$

$$Q_\pi(s,a) = \sum_{s'}\sum_{r} p(s',r|s,a)(r+\gamma V_\pi(s')) \tag{2}$$

$$V_\pi(s') = \sum_{a'} \pi(a'|s')Q(s',a') \tag{3}$$

Optimal policy

We can use iterate to find V and Q value of optimal policy if we have transition probability and reward

$$V^*(s) = \max_{a'} p(s,a,s')(r(s,a,s') + \gamma V^*(s')) \tag{4}$$

$$V_{k+1}(s) = \max_{a} \sum_{s'} p(s,a,s')(r(s,a,s') + \gamma V_k(s')) \tag{5}$$

$$Q_{k+1}(s,a) = \sum_{s'} p(s,a,s')(r(s,a,s') + \gamma \max_{a'} Q_k(s',a')) \tag{6}$$

$$\tag{7}$$

review MDP model

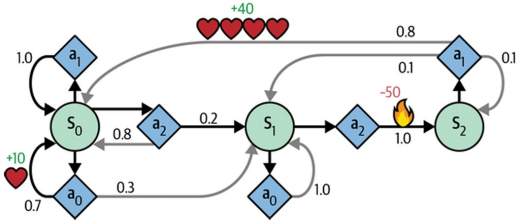


Figure 18-8. Example of a Markov decision process

Figure: MDP

```
transition_probabilities = [  # shape=[s, a, s']
    [[0.7, 0.3, 0.0], [1.0, 0.0, 0.0], [0.8, 0.2, 0.0]],
    [[0.0, 1.0, 0.0], None, [0.0, 0.0, 1.0]],
    [None, [0.8, 0.1, 0.1], None]
]
rewards = [  # shape=[s, a, s']
    [[+10, 0, 0], [0, 0, 0], [0, 0, 0]],
    [[0, 0, 0], [0, 0, 0], [0, 0, -50]],
    [[0, 0, 0], [+40, 0, 0], [0, 0, 0]]
]
possible_actions = [[0, 1, 2], [0, 2], [1]]
```

Figure: MDP

Iterate to find optimal policy

```python
transition_probabilities = [  # shape=[s, a, s']
    [[0.7, 0.3, 0.0], [1.0, 0.0, 0.0], [0.8, 0.2, 0.0]],
    [[0.0, 1.0, 0.0], None, [0.0, 0.0, 1.0]],
    [None, [0.8, 0.1, 0.1], None]
]
rewards = [  # shape=[s, a, s']
    [[+10, 0, 0], [0, 0, 0], [0, 0, 0]],
    [[0, 0, 0], [0, 0, 0], [0, 0, -50]],
    [[0, 0, 0], [+40, 0, 0], [0, 0, 0]]
]
possible_actions = [[0, 1, 2], [0, 2], [1]]
```

Figure: MDP

```python
gamma = 0.90  # the discount factor

for iteration in range(50):
    Q_prev = Q_values.copy()
    for s in range(3):
        for a in possible_actions[s]:
            Q_values[s, a] = np.sum([
                    transition_probabilities[s][a][sp]
                    * (rewards[s][a][sp] + gamma * Q_prev[sp].max())
                for sp in range(3)])
```

Figure: Iterate

Q learning without knowing MDP model

Content:

- The Temporal Diference Learning (TD Learning) algorithm is very similar to the Value Iteration algorithm, but tweaked to take into account the fact that the agent has only partial knowledge of the MDP
- In general we assume that the agent initially knows only the possible states and actions, and nothing more
- The agent uses an exploration policy—for example, a purely random policy—to explore the MDP, and as it progresses, the TD Learning algorithm updates the estimates of the state values based on the transitions and rewards that are actually observed

Q learning without knowing MDP model

$$Q_{k+1}(s,a) = \sum_{s'} p(s,a,s')(r(s,a,s') + \gamma \max_{a'} Q_k(s',a')) \tag{8}$$

$$Q_{k+1}(s,a) = (r + \gamma \max_{a'} Q_k(s',a')) \tag{9}$$

$$\tag{10}$$

Q learning without knowing MDP model

$$V_{k+1}(s) \leftarrow (1-\alpha)V_k(s) + \alpha(r + \gamma \cdot V_k(s'))$$
$$\text{or, equivalently:}$$
$$V_{k+1}(s) \leftarrow V_k(s) + \alpha \cdot \delta_k(s, r, s')$$
$$\text{with } \delta_k(s, r, s') = r + \gamma \cdot V_k(s') - V_k(s)$$

Figure: TD

epsilon-greedy

For each state s, this algorithm simply keeps track of a running average of the immediate rewards the agent gets upon leaving that state, plus the rewards it expects to get later (assuming it acts optimally).

Monte Carlo vs TD Q learning

Update $Q(s,a)$

$Q(s,a) = Q(s,a) + \eta(target - Q(s,a))$

Monte Carlo

$target = G(t) = r(t+1) + \gamma r(t+2) + \dots$ Q is update every episode

TD

$target = G(t) = r(t+1) + \gamma \max_{a'} Q(s', a')$

Q is update every time step

Q is updated according to greedy policy instead of epsilon policy, training policy is different from the policy generate data, it is off-policy

n-step TD

$target = G(t) = r(t+1) + \gamma r(t+2) + \gamma^2 \max_{a(t+2)} Q(s(t+2), a(t+2))$

Bias variance trade off

MC

MC has low bias and high variance since every step has different return MC update Q after a full episode

TD

every step, TD use one actual return ans estimate rest, it has high bias and low variance TD update Q every step

Q learning which update per step

```
Input: env, epsilon_num
Use a two-dimensional array Q(|S|, |A|) to represent the Q-function. Randomly initialize the Q array
for iterterate in range(0, epsilon_num):
        s, done = env.reset()
        while not done
                a=chose_action(Q,s) %$\pi(a|s)$
                s',r, done= env.step(a)
                $y=r+\gamma \max\limits_{a'} Q(s',a')$
                $Q(s,a)=Q(s,a)+ \eta (y-Q(s,a))$
                 % (y-Q(s,a) is Tempral Ddifference
                 s=s'
```

Policy improvement which update after many episode

policy evaluation

we can interact with env many episode and sample return of each $Q(s,a)$
$Q_\pi(s,a) = \frac{1}{N}\sum(Q(s,a))$

epsilon-greedy

epsilon-greedy allow us to explore pair (s,a) to get a lot data
if random() $<$ epsilon
return a random action
else
return $argmax_a(Q(s,a))$

Policy improvement which update after many episode

π is initialized as random policy

do following loop

step1:find $Q_\pi(s, a)$
step2: for each state s, its policy is $argmax_a(s, a)$

This method is too slow to use

there are two loop here, we have to run multiple episode to get valid statics of $Q(s, a)$.
evaluate Q by collect many sample

Q learning of Monte Carlo which update every episode

We update Q every episode

```
Randomly initialize the Q table
for iterterate in range(epsode_num):
        state,reward,action=[],[],[]
        s, done = env.reset()
        while not done
                a=chose_action(Q,s)
                    %$\pi(a|s)$
                s',r, done= env.step(a)
                state.append(s),reward.append(r),
                    action.append(a)
                s=s'
G=0
for s,r,a in reverse of state,reward,action
        G=r+G*\gamma G
        Q(s,a)=Q(s,a)+\eta (G-S(s,a))  %get
            average of Q(s,a)
```

Experiment

see `E:\lecture_code\AI\rl\18_reinforcement_learning.ipynb`.

What if state is continuous variable?

currently, state is discrete variable

Q(s,a) is a table

Table: Sample table with booktabs style

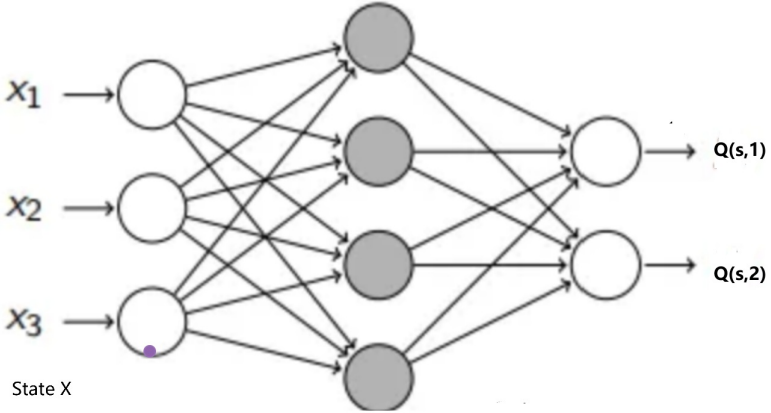
Action 1	Action 2	Action 3
Q(1, 1)	Q(1, 2)	Q(1, 3)
Q(2, 1)	Q(2, 2)	Q(2, 3)
Q(3, 1)	Q(3, 3)	Q(3, 3)

This does not work if we have infinite state or action

Approximate Q table with neural network:mechanism

Approximate Q table

we represent neural networks' function as $Q(s, a|\theta)$
update Q means to update θ which is $\theta = \theta + \eta(target - Q(s, a|\theta)) \bigtriangledown_\theta Q(s, a|\theta)$



Approximate Q table with neural network:theory

loss function

$Loss = (target - Q(s, a|\theta))^2$

$\bigtriangledown_\theta Loss = 2(target - Q(s, a|\theta)) \bigtriangledown_\theta Q(s, a|\theta)$

$\bigtriangledown_\theta Loss = 2(r + \max_{a'} Q(s', a') - Q(s, a|\theta)) \bigtriangledown_\theta Q(s, a|\theta)$

Here we assume target is a fixed value. but actually it is related with θ

experiment

See `E:\lecture_code\AI\rl\Q_Learning_Cartpole.ipynb`.

DQN

Content

This section provides details of MDP.

Content:

- basic structure
- replay buffer
- target networks
- update epsilon

Decreasing epsilon in DQN:necessary

Necessary to decrease epsilon in the epsilon-greedy algorithm

- exploration: when the train is started, the Q is wrong since Q is initialized randomly. So we need to explore Q space as much as possible by set epsilon a larger value.
- exploitation: at end of training, Q become accurate since it have collect enough data. agent can make optimal decision. we need set epsilon to a small value
- solution: decrease epsilon over time. actually it is a bandit problem.

Decreasing epsilon in DQN:working mechanism

many way to decrease epsilon, $\epsilon(t) = \frac{a}{b}$.

Decrease ϵ in DQN

We decrease ϵ linearly first, then fix it be a constant value

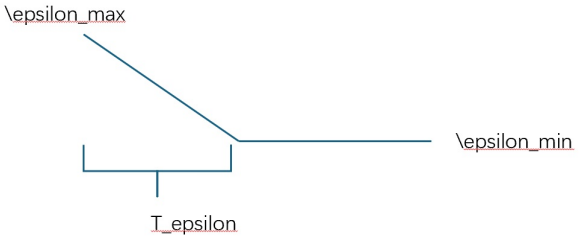


Figure: decrease epsilon in DQN

Replay buffer:Necessay

- Sequential Correlation in Online Learning: In traditional online learning, experiences are highly correlated since they are generated consecutively by the agent. Training a neural network on these sequential samples can lead to inefficient learning and divergence.
- Non-IID Data: Neural networks perform best when trained on independent and identically distributed (IID) data. However, in reinforcement learning, observations depend on previous actions and states, making the data highly non-IID.
- Inefficient Data Usage: Without reusing past experiences, each experience is used only once, leading to inefficient learning and requiring more interactions with the environment.

Replay buffer: working mechanism

definition of replay buffer

replay buffer is a memory to store experience collected at each step.
each experience is a tuple $s, a, s', r, done$ which represent currents state, action, next state, reward and done

Function of replay buffer

- Experience Collection: As the agent interacts with the environment, experiences (s, a, r, s', d) are stored in the replay buffer
- Sampling for Training: Instead of using the most recent experience for updating the Q-network, a mini-batch of experiences is sampled randomly from the buffer.
- Buffer Management: If the buffer reaches its maximum capacity, the oldest experiences are removed to make room for new ones..

Replay buffer: benift

- By storing past experiences and sampling randomly, the replay buffer ensures that updates are less correlated, helping the neural network generalize better.
- Instead of discarding each experience after one update, the replay buffer allows experiences to be reused multiple times, reducing the number of interactions needed with the environment.
- Using a mini-batch from past experiences leads to more stable gradient updates, reducing the variance in learning and preventing drastic fluctuations in policy updates.
- Since the agent learns from stored past experiences, the learning process is off-policy, meaning it can learn from experiences generated by different policies. This allows for greater flexibility in training.

Replay buffer: challenge

- Large replay buffers require significant memory, especially in high-dimensional environments like image-based RL tasks.
- Older experiences may become less relevant if the environment dynamics or policy changes significantly.
- While uniform sampling is simple, it may not be optimal. More sophisticated sampling strategies (like PER - prioritized experience replay) can introduce additional hyperparameters and computational complexity.

Replay buffer: limitation

Decrease ϵ in DQN

A lot redundancy in memory when state is picture, especially the state is combination of a few sequential picture to represent motion of object.

(s1, a1, **s2**, r2, d2)

(**s2**, a2, **s3**, r3, d3)

(**s3**, a3, s4, r4, d4)

([o1, o2, o3, o4], a4, [o2, o3, o4, o5], r5, d5)

([o2, o3, o4, o5], a5, [o3, o4, o5, o6], r6, d6)

Replay buffer: usage in the training

Decrease ϵ in DQN

In the context of replay buffer, collect data and training Q is separated.
In the traditional Q learning, the experience is used to update Q as soon as it is collected.

```
Q = random init
for i in range(training_steps):
  a = epsilon_greedy(Q, s)
  s2, r, d = env.step(a)
  buffer.store(s, a, r, s2, d)

  if i % train_every == 0
    batch = buffer.sample_batch()

    target = batch.r + gamma * max(Q(batch.s2, :)) * (1 - batch.d)
    pred = Q(batch.s)

    do gradient descent for (target - pred)**2

  s = s2
```

target network: necessary

review loss function

$Loss = (target - Q(s, a|\theta))^2$

$\bigtriangledown_\theta Loss = 2(r + \max_{a'} Q(s', a'|\theta) - Q(s, a|\theta)) \bigtriangledown_\theta Q(s, a|\theta)$

Here we assume target is related with θ. We use same network to calculate target and $Q(s, a)$

moving target

As we update Q, target is also moving which is hard to train.

positive feed

target is calculate by network we are updating. This may introduce positive feed loop which erratic Q value

overshooting bias

We use $\max$ operation even the Q value is noisy

target network: usage details

two networks

- online network θ_{online}: used to predict $Q(s, a; \theta_{online})$.
- target network θ_{target}: used to produce $Q(s', a'; \theta_{target})$.

====

sync two networks after a few steps

- in initialize stage, $\theta_{online} = \theta_{target}$
- in train stage, copy value of $\theta_{online})$ to $\theta_{target})$ after a few steps.

comments

$\theta_{target})$ will maintains static between update steps and it is old version of $\theta_{online})$. Thus the target is fixed if input is same.

target network: improvement

smooth averaging update

$\theta_{target} = \tau\theta_{target} + (1-\tau)\theta_{online}$ w.

double DQN aiming to sole over-estimate bias

- use online networks to select action of s', $select = \arg\max_{a'} Q(s', a'; \theta_{online})$
- $target = r + \gamma Q(s', select; \theta_{target})$.

DQN summary

- linearly decaying epsilon
- replay buffer
- target network
- training every few steps

work together

These features work together to enhance stability, efficiency, and learning performance. Proper tuning is crucial for effectively training deep RL agents

Experiment

see `E:\lecture_code\AI\rl\DQN_Learning_Cartpole.ipynb`.